

Project Management System

Vision Statement

To empower teams of all sizes to deliver projects more efficiently by providing an intuitive, centralized platform that streamlines task coordination, enhances real-time collaboration, and offers actionable insights—turning complex workflows into clear, manageable actions.

Primary Objectives

To achieve the vision, the system will:

- Centralize project communication and tracking to eliminate scattered tools and reduce context switching.
 - Automate task assignment and progress updates to minimize manual oversight and keep projects moving forward.
 - Provide real-time visibility into project status, team workload, and deadlines for all stakeholders.
 - Facilitate seamless collaboration through integrated discussions, file sharing, and activity feeds.
 - Deliver actionable analytics that help identify bottlenecks, forecast timelines, and optimize resource allocation.
 - Ensure system accessibility and responsiveness across devices to support remote and hybrid teams.
 - Maintain data security and role-based permissions to protect sensitive project information.
 - Design for scalability and ease of use to accommodate team growth without increasing administrative overhead.
-

Target Users and Core Use Cases

Target Users

1. Project Managers
2. Team Members/Contributors

Core Use Cases

The MVP focuses on two user types only: **Project Managers** (primary user who creates and manages projects) and **Team Members** (secondary user who completes assigned tasks).

For the MVP, the absolute minimum features must create a complete, usable loop between a Project Manager (PM) and a Team Member (TM). Below are the essential features and their single core purposes.

Project Manager (PM) Features

1. **Create Projects & Tasks**
 - Single Core Purpose: To establish the basic structure and scope of work.
2. **Assign Tasks to Team Members**
 - Single Core Purpose: To distribute ownership and clarify who is responsible for what.
3. **View a Task Status Dashboard**
 - Single Core Purpose: To monitor overall progress without manual check-ins.

Team Member (TM) Features

1. **View an "Assigned Tasks" List**
 - Single Core Purpose: To see a personal, clear list of what to work on.
2. **Update Task Status**
 - Single Core Purpose: To communicate progress back to the PM and team (e.g., To Do → In Progress → Done).
3. **Add Comments to a Task**
 - Single Core Purpose: To ask questions or provide updates within the context of the work.

Essential MVP requirements

Authentication

- **Email & Password Sign-up/Login:** Basic form to create and authenticate users.
- **JWT Session Tokens:** Secure method to keep users logged in across requests.
- **Protected Route Middleware:** Code to block unauthorized access to app features.

User Onboarding

- **Role Assignment (PM/TM):** Designate user as Project Manager or Team Member upon sign-up.
- **Immediate Core App Access:** Redirect user to their main dashboard after login.

Admin / Backend Essentials

- **Environment Variables File:** Secure storage for database credentials and API keys.
- **Manual Database Access for Critical Issues:** Ability to fix user accounts or roles directly via the database if needed.

Core User Features (Functional MVP)

- **PM:** Create Project & Tasks — Establish the basic unit of work and its components.
- **PM:** Assign Tasks to Members — Designate responsibility and distribute work.
- **PM:** View Project Task Status Dashboard — Monitor overall progress visually.
- **TM:** View Personal Assigned Tasks List — See a clear, filtered list of owned work.
- **TM:** Update Task Status (To Do/In Progress/Done) — Communicate progress back to the team.
- **TM:** Add Comments to Tasks — Enable contextual discussion and updates on work items.

MVP Functional Specification: Core Essentials

1. User Stories & Acceptance Criteria

The user stories below define the functional expectations for Project Managers (PM) and Team Members (TM). These stories and acceptance criteria establish the behaviors the system must support.

US-AUTH-01: User Registration & Login

As a new user, I want to create an account and log in, so that I can access the system.

AC 1: GIVEN I am on the sign-up page, WHEN I enter a valid email and password, THEN a new account is created, and I am logged in.

AC 2: GIVEN I have an account, WHEN I enter my correct email and password on the login page, THEN I am logged in and redirected to my role-specific dashboard.

US-PM-01: Create Project & Task

As a Project Manager, I want to create a new project and add tasks to it, so that I can define the work scope.

AC 1: GIVEN I am logged in as a PM, WHEN I create a project with a title, THEN it appears in my dashboard.

AC 2: GIVEN I am viewing a project, WHEN I add a task with a title, THEN the task appears in the project's "To Do" column.

US-PM-02: Assign Task

As a PM, I want to assign a task to a Team Member, so that responsibility is clear.

AC: GIVEN I am viewing a "To Do" task, WHEN I select a TM from the list and assign it, THEN the task shows that TM as the assignee.

US-PM-03: View Task Dashboard

As a PM, I want to see all tasks in columns by status, so that I can track progress.

AC: GIVEN I am a PM, WHEN I view my project dashboard, THEN I see tasks grouped in “To Do”, “In Progress”, and “Done” columns.

US-TM-01: View My Assigned Tasks

As a Team Member, I want to see a list of tasks assigned to me, so that I know what to work on.

AC: GIVEN I am logged in as a TM, WHEN I go to my dashboard, THEN I see only tasks where I am the assignee.

US-TM-02: Update Task Status

As a TM, I want to change the status of my task, so that I can show progress.

AC 1: GIVEN I am the assignee of a “To Do” task, WHEN I mark it “In Progress”, THEN its status updates.

AC 2: GIVEN I am the assignee of an “In Progress” task, WHEN I mark it “Done”, THEN its status updates and it moves to the “Done” column.

US-TM-03: Comment on Task

As a TM, I want to add a text comment to my task, so that I can ask questions or give updates.

AC: GIVEN I am viewing a task assigned to me, WHEN I post a comment, THEN it appears in the task’s comment list with my name and timestamp.

2. Task Lifecycle & State Transitions

To support the user stories above, the system enforces a defined task lifecycle and controlled state transitions.

State Transition Table

- **Row 1**
 - Current Status: To Do
 - Allowed New Status: In Progress
 - Who Can Do It: Assigned TM Only
 - Condition: Task must have an assignee.
- **Row 2**
 - Current Status: In Progress
 - Allowed New Status: Done
 - Who Can Do It: Assigned TM Only
 - Condition: —
- **Row 3**
 - Current Status: Done
 - Allowed New Status: In Progress
 - Who Can Do It: Assigned TM or PM
 - Condition: For rework/corrections.
- **Row 4**
 - Current Status: Any
 - Allowed New Status: Reassigned

Who Can Do It: PM Only
Condition: Changes assignee, not status.

State Diagram Explanation

A task starts as To Do. Only the assigned Team Member can move it forward to In Progress and then to Done. Moving backwards from In Progress to To Do is not allowed. A Done task can be moved back to In Progress if needed. A Project Manager can reassign a task at any point without changing its status.

3. Happy Path & Error Flows

This section illustrates expected system behavior during normal operation and when invalid actions are attempted.

3.1 Happy Path: Completing a Task

1. PM Action: PM creates "Task X" in Project A. Status: To Do, Assignee: None.
2. PM Action: PM assigns "Task X" to Team Member Jane.
3. TM Action: Jane views her dashboard and sees "Task X".
4. TM Action: Jane changes "Task X" status from To Do → In Progress.
5. TM Action: Jane adds a comment: "Starting work on this."
6. TM Action: Jane finishes and changes status from In Progress → Done.
7. System Action: Task moves to the "Done" column on the PM's dashboard.

3.2 Common Error Flows

EF-1: Wrong User Updates Status

TM Bob tries to update the status of a task assigned to TM Jane.
System Prevents: Shows error: "You can only update tasks assigned to you."

EF-2: Invalid Status Change

TM Jane tries to change a task from In Progress back to To Do.
System Prevents: Shows error: "Cannot move task back to 'To Do'."

EF-3: Assigning to Non-Existent User

PM tries to assign a task by typing a name.
System Prevents: Only allows selection from a dropdown of registered TMs.

4. Business Rules

The following business rules formalize constraints referenced throughout the specification.

BR-1: Task Creation

Description: A task must have a **title**.
Applies To: System

BR-2: One-Way Progression

Description: Status can move: To Do → In Progress → Done. "In Progress" cannot revert

to “To Do”, but “Done” can revert to “In Progress”

Applies To: Task State

BR-3: Assignee-Based Updates

Description: Only the user listed as the **assignee** can change a task’s status.

Applies To: Permissions

BR-4: Reassignment

Description: A PM can change the **assignee** at any time. This does not change the **status**.

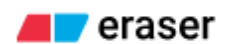
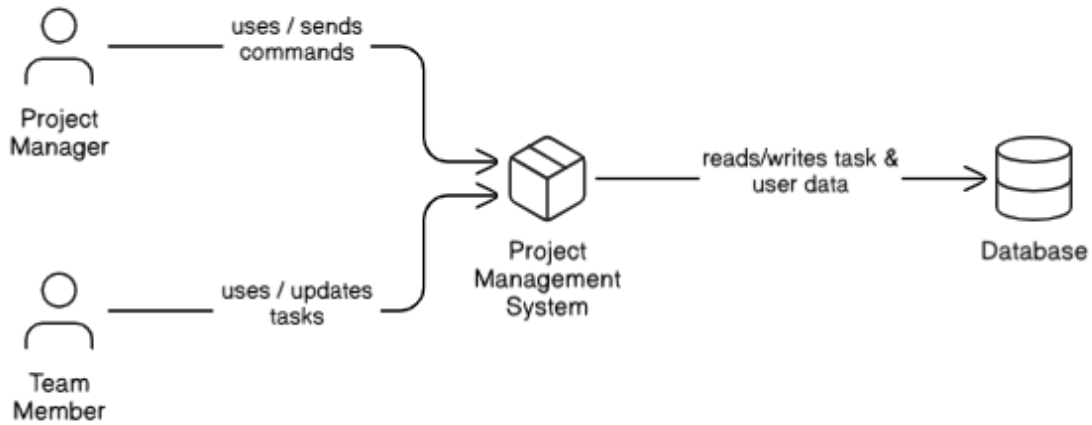
Applies To: Permissions

BR-5: Comment Ownership

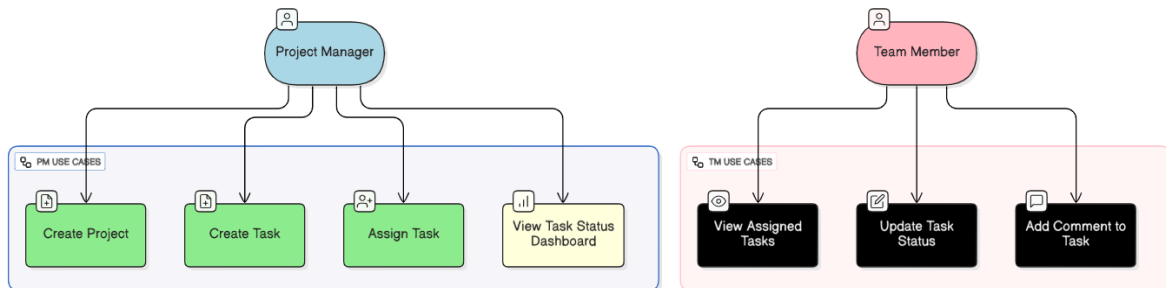
Description: Comments are permanently attached to the task and show the author’s name and time.

Applies To: Data

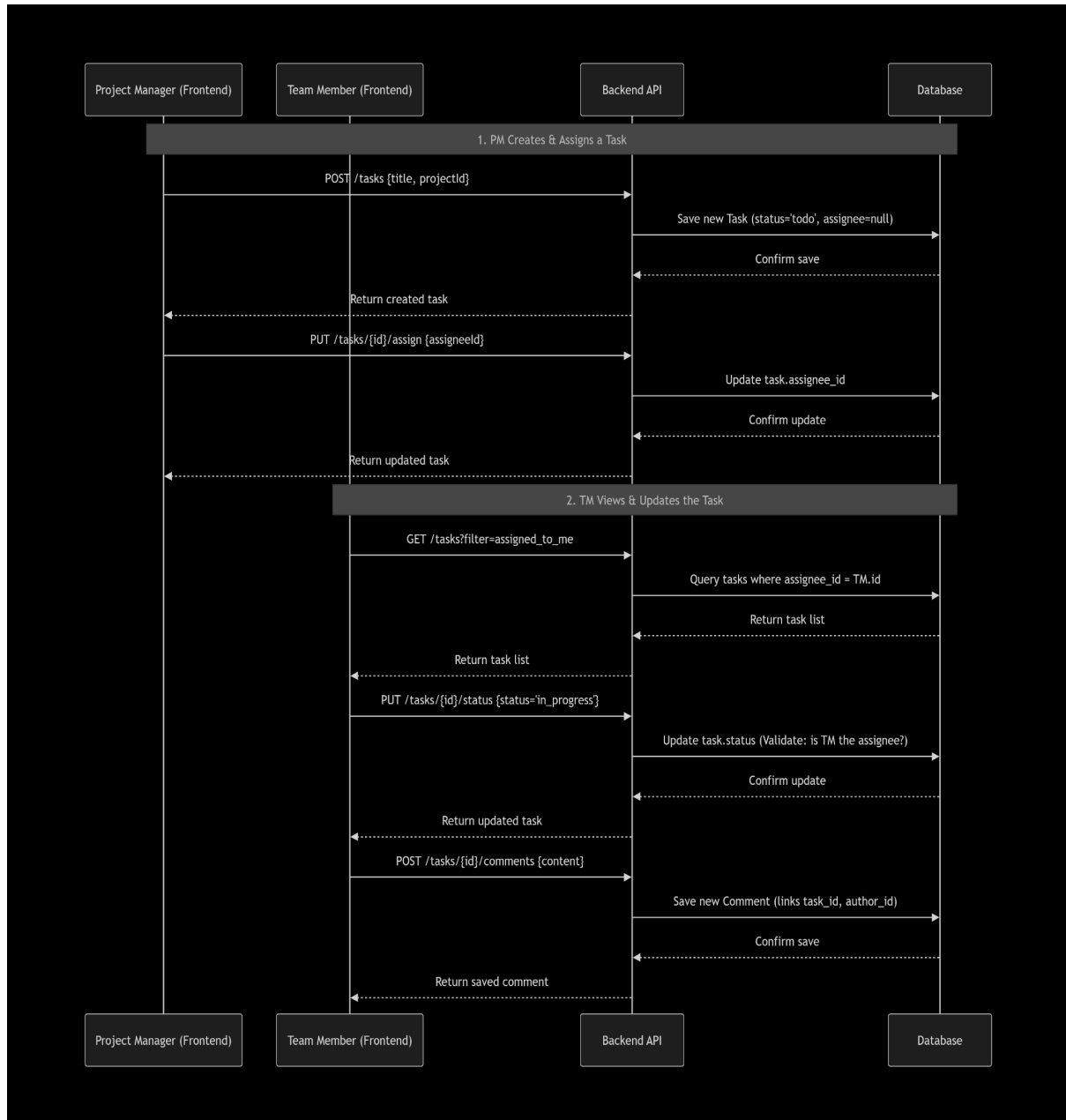
1 System Context Diagram



2 Use Case Diagram



3 Sequence Diagram: Task Assignment & Update Flow



Data models

Based on MVP specification, the system requires four core data models. The relationships between these models enable the complete PM-to-TM task loop.

1. User Model

- * **Stores:** System account information and role.
- * **Core Fields:** `id`, `name`, `email`, `password_hash`, `role` (e.g., `project_manager`, `team_member`).
- * **Relationships:**
 - * A **User** *owns* many **Projects** (One-to-Many).
 - * A **User** *is assigned to* many **Tasks** (One-to-Many).
 - * A **User** *writes* many **Comments** (One-to-Many).

2. Project Model

- * **Stores:** The primary container for organizing work.
- * **Core Fields:** `id`, `title`, `description`, `owner_id` (foreign key to User).
- * **Relationships:**
 - * A **Project** *is owned by* one **User** (the Project Manager) (Many-to-One).
 - * A **Project** *contains* many **Tasks** (One-to-Many).

3. Task Model (The Central Entity)

- * **Stores:** The fundamental unit of work.
- * **Core Fields:** `id`, `title`, `description`, `status` (e.g., `todo`, `in_progress`, `done`), `assignee_id` (foreign key to User), `project_id` (foreign key to Project).
- * **Relationships:**
 - * A **Task** *belongs to* one **Project** (Many-to-One).
 - * A **Task** *is assigned to* one **User** (Team Member) (Many-to-One).
 - * A **Task** *has* many **Comments** (One-to-Many).

4. Comment Model

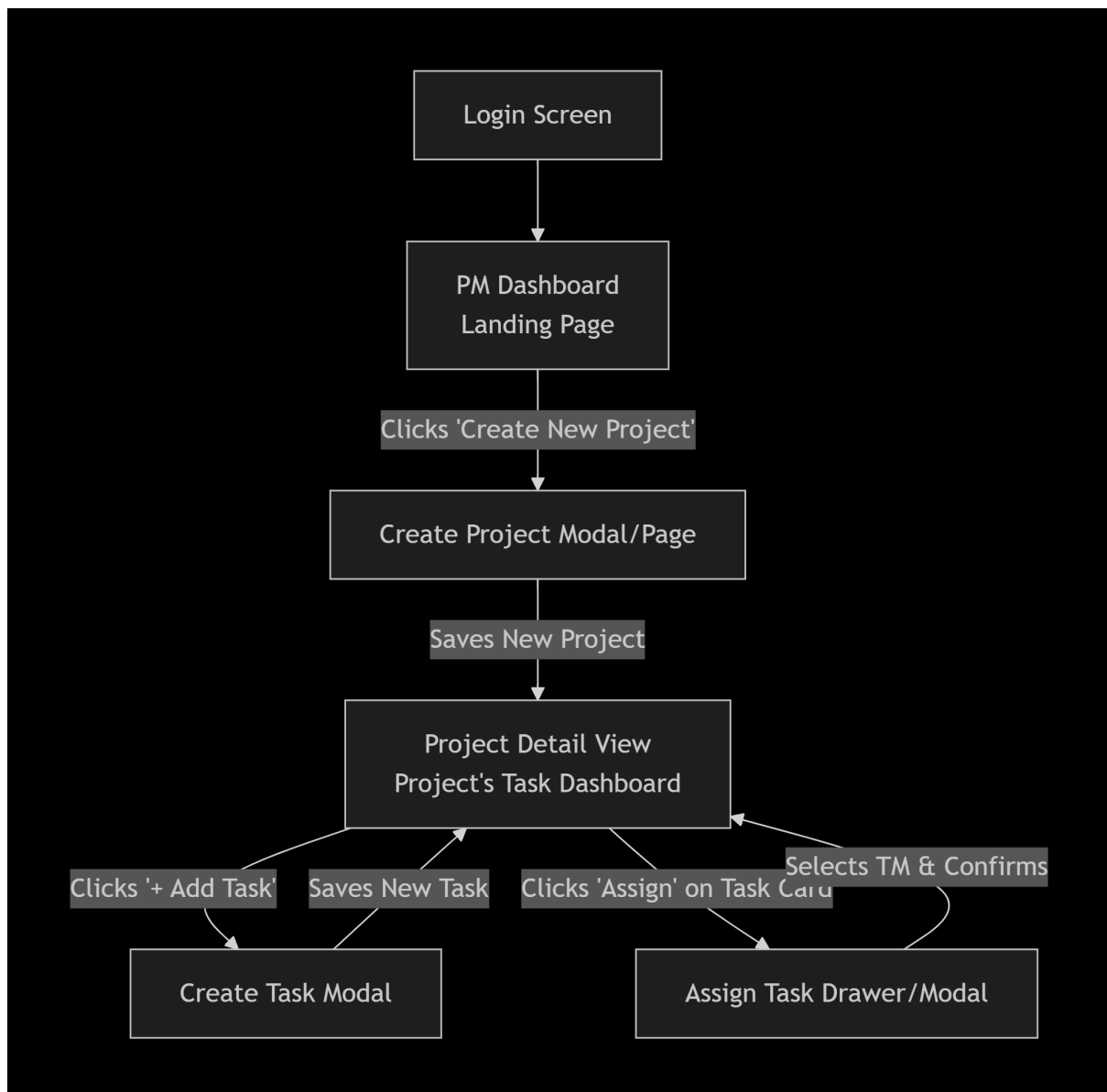
- * **Stores:** Contextual updates or questions on a task.
- * **Core Fields:** `id`, `content`, `task_id` (foreign key to Task), `author_id` (foreign key to User), `createdAt`.
- * **Relationships:**
 - * A **Comment** *belongs to* one **Task** (Many-to-One).
 - * A **Comment** *is written by* one **User** (Many-to-One).

💡 Relationship Summary

This structure creates the following flows:

1. **PM Creates Work:** A **User** (with role **project_manager**) creates a **Project** and its **Tasks**.
2. **PM Assigns Work:** A **Task** is linked to a **User** (with role **team_member**) via **assignee_id**.
3. **TM Executes & Communicates:** A **User** (team member) updates their assigned **Task.status** and adds **Comments** to it.

Simple prototype or a very basic wireframe of the application



Essential MVP Wireframes

1. Login / Sign-up

- **Purpose:**
The single entry point for all users to authenticate and declare their role.
- **Key Components:**
 - Email field
 - Password field
 - Role selector (PM / TM)
 - Login button
 - Sign-up button

2. PM Dashboard

- **Purpose:**
The Project Manager's landing page to view all projects and initiate new ones.
- **Key Components:**
 - "Create New Project" button
 - List of existing project cards
 - Project title
 - Task summary

3. Project Detail & Task Board

- **Purpose:**
The Project Manager's workspace to manage a single project, including task creation and assignment.
- **Key Components:**
 - Project title
 - "+ Add Task" button
 - Task board with columns:
 - To Do
 - In Progress
 - Done
 - Task cards with "Assign" button

4. TM Dashboard (My Tasks)

- **Purpose:**
The Team Member's main view showing only tasks assigned to them.
- **Key Components:**
 - Task board with columns:
 - To Do
 - In Progress
 - Done
 - Tasks filtered to show only those where the user is the assignee

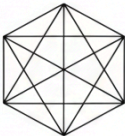
5. Task Detail

- **Purpose:**
A shared screen for viewing task details, updating status, and adding comments.
- **Key Components:**
 - Task title
 - Task description
 - Status update button or dropdown
 - Comment input field
 - History of previous comments

How These Screens Connect

1. A user logs in through **Login / Sign-up** and is redirected to their role-specific dashboard:
 - PM → **PM Dashboard**
 - TM → **TM Dashboard (My Tasks)**
2. A Project Manager creates a project from the **PM Dashboard**, which opens the **Project Detail & Task Board**.
3. From the **Project Detail & Task Board**, the Project Manager creates and assigns tasks.
4. The assigned Team Member sees the task appear on their **TM Dashboard (My Tasks)**.
Selecting the task opens the **Task Detail** screen to update status and add comments.
5. The Project Manager sees task status updates reflected in real time on the **Project Detail & Task Board**.

1. Login / Sign-up Screen



PROJECT FLOW

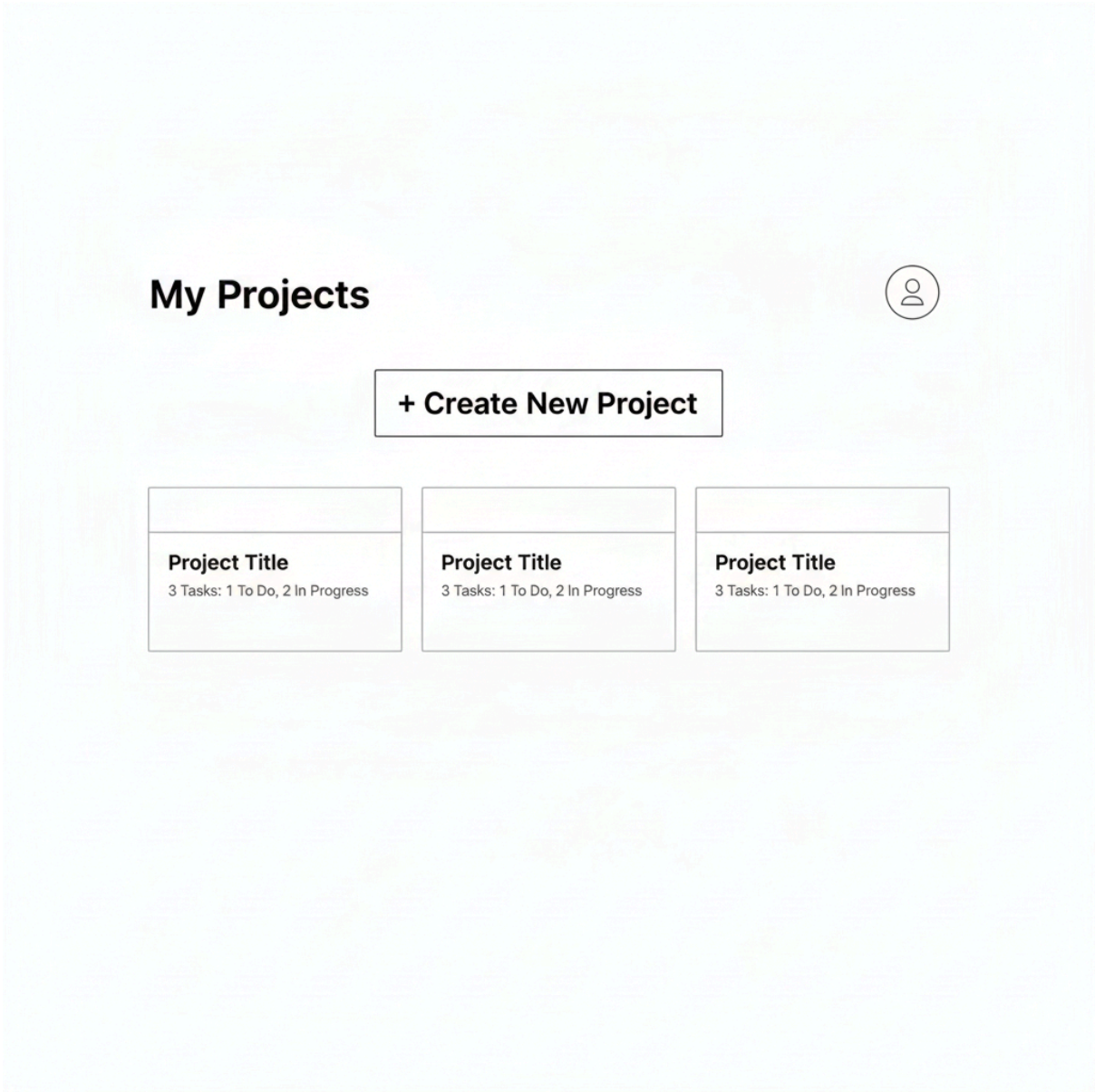
Email	Password
	Team Member

I am a:

▼

Log In	Sign Up
---------------------------------------	--

2. PM Dashboard (Project List)



3. Project Detail & Task Board

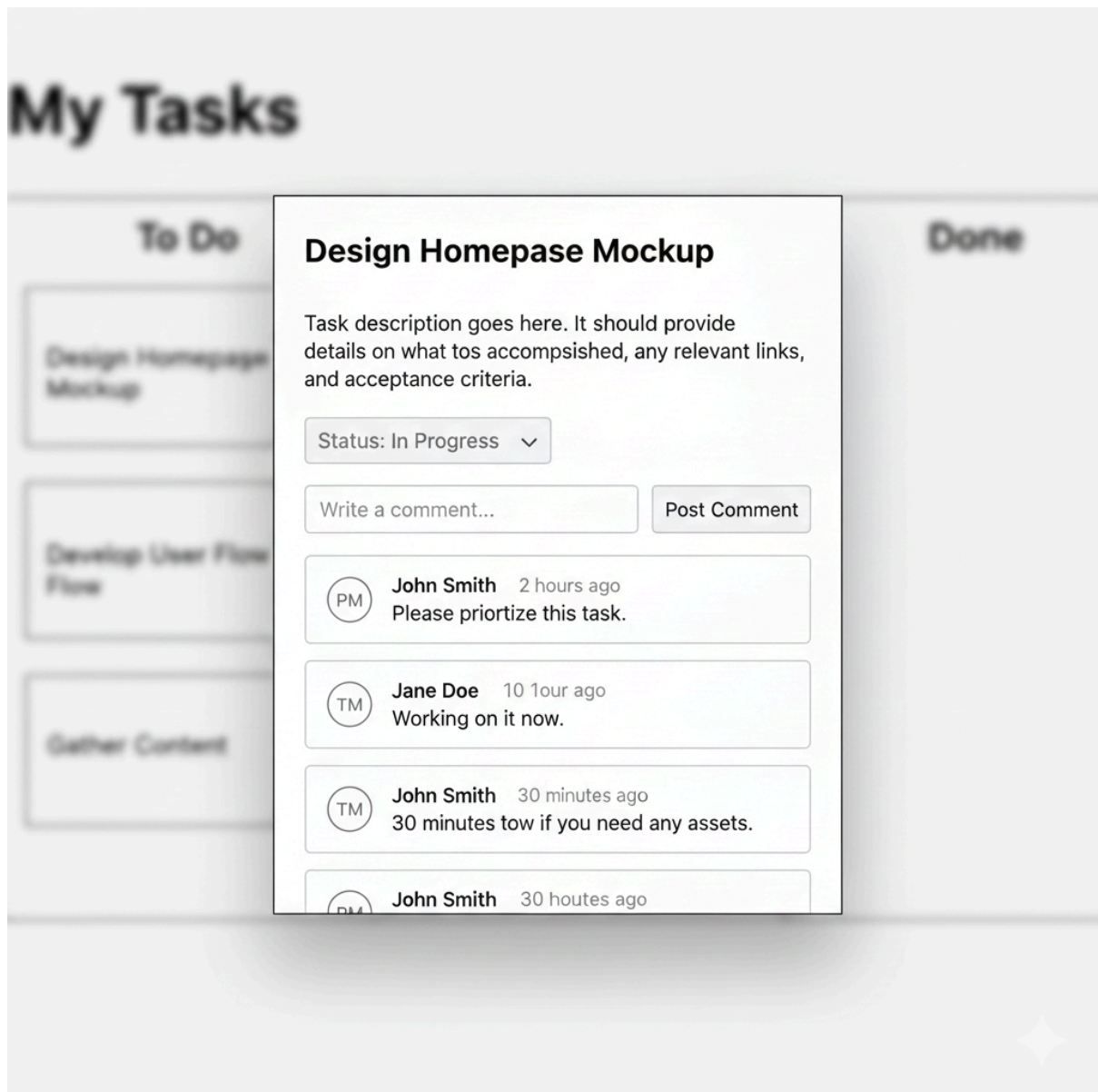


4. TM Dashboard (My Tasks Board)

My Tasks

To Do	In Progress	Done
Design Homepage Mockup		
Develop User Flow Flow	Implement Login Screen	
Gather Content		

5. Task Detail View



- **Technology Stack:**

- **Backend Framework:**
 - Node.js
 - Express.js
- **Database:**
 - MySQL
- **Frontend:**
 - React.js
- **Styling:**
 - CSS
- **Version Control:**
 - Git

Database design

Table 1: **users**

- **Relations:**
 - With **projects**: One-to-Many on **owner_id**
 - With **tasks**: One-to-Many on **assignee_id**
 - With **comments**: One-to-Many on **author_id**
- **Table fields:**
 - **id** (Primary Key) INT
 - **email** (Unique) VARCHAR(255)
 - **password_hash** VARCHAR(255)
 - **name** VARCHAR(255)
 - **role** ENUM('project_manager', 'team_member')
 - **created_at** TIMESTAMP

Table 2: **projects**

- **Relations:**
 - With **users**: Many-to-One on **owner_id**
 - With **tasks**: One-to-Many on **project_id**
- **Table fields:**
 - **id** (Primary Key) INT
 - **title** VARCHAR(255)
 - **description** TEXT
 - **owner_id** (Foreign Key) INT
 - **created_at** TIMESTAMP

Table 3: **tasks**

- **Relations:**
 - With **projects**: Many-to-One on **project_id**
 - With **users**: Many-to-One on **assignee_id**
 - With **comments**: One-to-Many on **task_id**
- **Table fields:**
 - **id** (Primary Key) INT
 - **title** VARCHAR(255)
 - **description** TEXT
 - **status** ENUM('todo', 'in_progress', 'done')
 - **project_id** (Foreign Key) INT
 - **assignee_id** (Foreign Key) INT
 - **created_at** TIMESTAMP
 - **updated_at** TIMESTAMP

Table 4: **comments**

- **Relations:**
 - With **tasks**: Many-to-One on **task_id**
 - With **users**: Many-to-One on **author_id**
- **Table fields:**
 - **id** (Primary Key) INT
 - **content** TEXT
 - **task_id** (Foreign Key) INT
 - **author_id** (Foreign Key) INT
 - **created_at** TIMESTAMP

API Endpoints

API Endpoint Specification

1) Register User

- **Method:** POST
- **Path:** **/api/auth/register**
- **Purpose:** Creates a new user account with a specific role (Project Manager or Team Member).

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
```

Request Example:

```
POST /api/auth/register HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
{
  "email": "alex@example.com",
  "password": "securePassword123",
  "name": "Alex Chen",
  "role": "team_member"
}
```

Successful Response (201 Created):

```
Content-Type: application/json
Body:
{
  "message": "User created successfully",
  "user": {
    "id": 5,
    "email": "alex@example.com",
    "name": "Alex Chen",
    "role": "team_member"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Status Codes:

- 201: User successfully created.
- 400: Bad request (e.g., missing fields, invalid email, role not 'project_manager' or 'team_member').
- 409: Conflict (email already exists).
- 500: Internal server error.

2) Login User

- **Method:** POST
- **Path:** `/api/auth/login`
- **Purpose:** Authenticates a user and returns a JWT token for subsequent requests.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
```

Request Example:

```
POST /api/auth/login HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
```

```
{
  "email": "alex@example.com",
  "password": "securePassword123"
}
```

Successful Response (200 OK):

```
Content-Type: application/json
Body:
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 5,
    "email": "alex@example.com",
    "name": "Alex Chen",
    "role": "team_member"
  }
}
```

Status Codes:

- 200: Login successful.
- 401: Unauthorized (invalid credentials).
- 400: Bad request (missing fields).
- 500: Internal server error.

3) Create Project

- **Method:** POST
- **Path:** `/api/projects`
- **Purpose:** Creates a new project. The authenticated user becomes the project owner.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
POST /api/projects HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>

{
  "title": "Website Redesign",
  "description": "Redesign the company homepage and main product
pages."
}
```

Successful Response (201 Created):

```
Content-Type: application/json
Body:
{
  "id": 12,
  "title": "Website Redesign",
  "description": "Redesign the company homepage and main product
pages.",
  "owner_id": 1,
  "created_at": "2023-10-26T10:30:00.000Z"
}
```

Status Codes:

- 201: Project successfully created.
- 400: Bad request (e.g., missing title).
- 401: Unauthorized (missing or invalid token).
- 500: Internal server error.

4) List Projects

- **Method:** GET
- **Path:** `/api/projects`
- **Purpose:** Retrieves a list of all projects owned by the authenticated Project Manager, including a summary of task counts.

Headers:

```
Host: localhost:8080
Accept: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
GET /api/projects HTTP/1.1
Host: localhost:8080
Accept: application/json
Authorization: Bearer <JWT_TOKEN>
```

Successful Response (200 OK):

```
Content-Type: application/json
Body:
{
  "projects": [
    {
      "id": 12,
      "title": "Website Redesign",
      "task_summary": {
        "todo": 2,
        "in_progress": 1,
        "done": 5
      }
    },
    {
      "id": 13,
      "title": "Q4 Planning",
      "task_summary": {
        "todo": 4,
        "in_progress": 0,
        "done": 0
      }
    }
  ]
}
```

Status Codes:

- 200: Request successful.

- 401: Unauthorized (missing or invalid token).
- 403: Forbidden (user is not a Project Manager).
- 500: Internal server error.

5) Create Task

- **Method:** POST
- **Path:** `/api/projects/:projectId/tasks`
- **Purpose:** Creates a new task within a specified project.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
POST /api/projects/12/tasks HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>

{
  "title": "Design Homepage Mockup",
  "description": "Create initial design concepts for the new
homepage."
}
```

Successful Response (201 Created):

```
Content-Type: application/json
Body:
{
  "id": 44,
  "title": "Design Homepage Mockup",
  "description": "Create initial design concepts for the new
homepage.",
  "status": "todo",
  "project_id": 12,
  "assignee_id": null,
  "created_at": "2023-10-26T10:35:00.000Z"
```

```
}
```

Status Codes:

- 201: Task successfully created.
- 400: Bad request (e.g., missing title, invalid projectId).
- 401: Unauthorized (missing or invalid token).
- 403: Forbidden (user does not own the project).
- 404: Project not found.
- 500: Internal server error.

6) Assign Task

- **Method:** PUT
- **Path:** `/api/tasks/:taskId/assign`
- **Purpose:** Assigns or reassigns a specific task to a Team Member. Only the Project Manager who owns the task's project can perform this action.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
PUT /api/tasks/44/assign HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>

{
  "assignee_id": 5
}
```

Successful Response (200 OK):

```
Content-Type: application/json
Body:
{
  "message": "Task assigned successfully",
}
```

```
"task": {  
  "id": 44,  
  "title": "Design Homepage Mockup",  
  "assignee_id": 5,  
  "assignee_name": "Alex Chen"  
}
```

Status Codes:

- 200: Task successfully assigned.
- 400: Bad request (e.g., missing assignee_id, assignee is not a Team Member).
- 401: Unauthorized (missing or invalid token).
- 403: Forbidden (user is not the PM owner of the task's project).
- 404: Task or assignee not found.
- 500: Internal server error.

7) List Team Members

- **Method:** GET
- **Path:** `/api/users`
- **Purpose:** Retrieves a list of users, typically filtered to only Team Members, for use in assignment dropdowns.

Headers:

```
Host: localhost:8080  
Accept: application/json  
Authorization: Bearer <JWT_TOKEN>
```

Query Parameters (if any):

- **role:** Filters users by their role. Use `role=team_member` to get list for assignment.

Request Example:

```
GET /api/users?role=team_member HTTP/1.1  
Host: localhost:8080  
Accept: application/json  
Authorization: Bearer <JWT_TOKEN>
```

Successful Response (200 OK):

```
Content-Type: application/json
```

```
Body:
{
  "users": [
    {
      "id": 5,
      "name": "Alex Chen",
      "email": "alex@example.com"
    },
    {
      "id": 8,
      "name": "Jamie Smith",
      "email": "jamie@example.com"
    }
  ]
}
```

Status Codes:

- 200: Request successful.
- 401: Unauthorized (missing or invalid token).
- 500: Internal server error.

8) Get My Tasks

- **Method:** GET
- **Path:** `/api/tasks/me`
- **Purpose:** Retrieves all tasks assigned to the currently authenticated Team Member.

Headers:

```
Host: localhost:8080
Accept: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
GET /api/tasks/me HTTP/1.1
Host: localhost:8080
Accept: application/json
Authorization: Bearer <JWT_TOKEN>
```

Successful Response (200 OK):

```
Content-Type: application/json
```

```
Body:
{
  "tasks": [
    {
      "id": 44,
      "title": "Design Homepage Mockup",
      "status": "in_progress",
      "project_title": "Website Redesign"
    },
    {
      "id": 50,
      "title": "Fix Navigation Bug",
      "status": "todo",
      "project_title": "Q4 Planning"
    }
  ]
}
```

Status Codes:

- 200: Request successful.
- 401: Unauthorized (missing or invalid token).
- 403: Forbidden (user is not a Team Member).
- 500: Internal server error.

9) Update Task Status

- **Method:** PUT
- **Path:** `/api/tasks/:taskId/status`
- **Purpose:** Updates the status of a specific task. Must be performed by the assigned Team Member and follow allowed state transitions.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
PUT /api/tasks/44/status HTTP/1.1
Host: localhost:8080
Accept: application/json
```

```
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "status": "in_progress"
}
```

Successful Response (200 OK):

```
Content-Type: application/json
Body:
{
  "message": "Status updated",
  "task": {
    "id": 44,
    "title": "Design Homepage Mockup",
    "status": "in_progress",
    "updated_at": "2023-10-26T11:40:00.000Z"
  }
}
```

Status Codes:

- 200: Status successfully updated.
- 400: Bad request (e.g., invalid status value, invalid state transition like moving "in_progress" back to "todo").
- 401: Unauthorized (missing or invalid token).
- 403: Forbidden (user is not the assigned Team Member for this task).
- 404: Task not found.
- 500: Internal server error.

10) Add Comment

- **Method:** POST
- **Path:** `/api/tasks/:taskId/comments`
- **Purpose:** Adds a text comment to a specific task. The authenticated user becomes the author.

Headers:

```
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>
```

Request Example:

```
POST /api/tasks/44/comments HTTP/1.1
Host: localhost:8080
Accept: application/json
Content-Type: application/json
Authorization: Bearer <JWT_TOKEN>

{
  "content": "Starting work on this now. Will share drafts by EOD."
}
```

Successful Response (201 Created):

```
Content-Type: application/json
Body:
{
  "id": 301,
  "content": "Starting work on this now. Will share drafts by EOD.",
  "author_id": 5,
  "author_name": "Alex Chen",
  "created_at": "2023-10-26T11:45:00.000Z"
}
```

Status Codes:

- 201: Comment successfully created.
- 400: Bad request (e.g., empty content).
- 401: Unauthorized (missing or invalid token).
- 404: Task not found.
- 500: Internal server error.